

EXPERIMENT - 3

Debugging C Programs

I. Objective

To understand the fundamentals of errors types and differentiating between bugs, try to solve these bugs with different editing and debugging tools.

2. Fundamentals of Errors and Debugging

2.1 Error:

A mistake or blunder made by the programmer in the source code.

Example: Writing $a + b$ when the intention was to multiply $a * b$.

2.2 Bug

A flaw or fault in a computer program that causes it to produce an incorrect or unexpected result, or to behave in unintended ways.

2.3 Debugging

The systematic process of finding, analyzing, and removing bugs or defects in a computer program.

3. Classifications of Errors in C

3.1 Syntax Errors

Code violates the grammatical rules of the C language, this type of error occurs.

Primary Cause: Missing semicolons, unmatched braces, typos in keywords.

Example: `int a = 10 (missing ; )`

3.2 Compilation /Type Errors

Operations performed on incompatible types or missing declarations.

Primary Cause: Assigning float string to int pointer, using undeclared variable.

Example: `int *p = "hello";`

3.3 Linker Errors

The compiler generates object files, but the linker cannot combine them.

Primary Cause: Function declared but not defined, missing libraries.

Example: undefined reference to 'main'

3.4 Runtime Errors

The program crashes or terminates abnormally during execution.

Primary Cause: Division by zero, accessing invalid memory addresses.

Example: `int a = 5 / 0;`

3.4 Runtime Errors

Code runs to completion without crashing, but output is incorrect.

Primary Cause: Wrong mathematical formula, incorrect loop condition.

Example: `for(i=0; i<5; i++)` instead of `<=5`

CASE-I :Syntax and Compiler Errors

A syntax error occurs at the exact moment the programmer writes code that violates the grammatical rules of the programming language.

```
//Example code(sample1.c)
#include <stdio.h>
int main()
{
    int a=10
    printf("%d\n",a);
    return 0;
}
```

a. Output & Compilation Command.

```
kaith@fedora:~/codes/exp_3$ cc sample1.c
sample1.c: In function 'main':
sample1.c:5:13: error: expected ',' or ';' before 'printf'
    5 |         printf("%d\n",a);
      |         ^~~~~~
kaith@fedora:~/codes/exp_3$
```

b. Breaking of error message

- We have a C program called sample1.c that contains a main function. Inside it, there's a printf statement trying to print an integer variable a .
- On line 5, character 9, the compiler expected to see a comma (,) or semicolon (;) right before the word printf.
- This means there is a mistake on the line ABOVE line 5 (line 4) that is confusing the compiler. The compiler is pointing to printf because it got lost trying to understand the previous line.

CASE-2 : Compiler Warnings (-Wall, -Wextra)

A warning indicates code that is syntactically legal, but potentially dangerous or suspicious. Warnings do not stop compilation, but ignoring them frequently leads to bugs at runtime.

//Example code(sample2.c)

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a;
```

```
    printf("Hello\n");
```

```
    return 0;
```

```
}
```

a. Output & Compilation Command.

```
kaith@fedora:~/codes/exp_3$ cc sample2.c
kaith@fedora:~/codes/exp_3$ ./a.out
Hello
kaith@fedora:~/codes/exp_3$
```

- Compiler is compiling the code without showing the errors.
- By adding -Wall -Wextra we can tell the compiler to flag the errors and show them in the execution part.
- -Wall: Enables a broad set of commonly useful compiler warnings.
- -Wextra: Enables additional warnings not covered by -Wall

b. Output after -Wall -Wextra

```
kaith@fedora:~/codes/exp_3$ cc -Wall -Wextra sample2.c
sample2.c: In function 'main':
sample2.c:4:13: warning: unused variable 'a' [-Wunused-variable]
    4 |         int a;
      |         ^
kaith@fedora:~/codes/exp_3$
```

CASE-3 : printf-Based Debugging (Logical bug)

We insert printf statements at strategic points in our code to print out variable values, messages, or execution flow while the program runs. Instead of using a debugger with breakpoints and step-through tools, we let the program tell us what it's doing by printing information to the terminal.

```
//Example code(sample3.c)
#include<stdio.h>
int main()
{
    int i,sum=0;
    for(i=0;i<5;i++)
    {
        printf("DEBUG:starting iteration i=%d\n",i);
        sum = sum+i;
        printf("DEBUG:updated sum =%d\n",sum);
    }
    printf("Final Sum=%d\n",sum);
    return 0;
}
```

a. Output

```
kaith@fedora:~/codes/exp_3$ cc sample3.c
kaith@fedora:~/codes/exp_3$ ./a.out
DEBUG:starting iteration i=0
DEBUG:updated sum =0
DEBUG:starting iteration i=1
DEBUG:updated sum =1
DEBUG:starting iteration i=2
DEBUG:updated sum =3
DEBUG:starting iteration i=3
DEBUG:updated sum =6
DEBUG:starting iteration i=4
DEBUG:updated sum =10
Final Sum=10
kaith@fedora:~/codes/exp_3$
```

- The loop is only executing till i=4 not for i=5 by printing the output on each stage we come to know that in the code itself the loop statement is not correct .
- To correct the code we need to edit the “i<5” to “i<=5”.

4. Introduction to GDB (GNU Debugger)

- GNU: A recursive acronym for "GNU's Not Unix!". It is an open-source operating system project initiated by Richard Stallman in 1983 to provide a completely free Unix-like system.
- GDB: GDB is a powerful command-line tool that lets you see what is happening inside your C/C++ program while it runs—or what happened when it crashed.

Quick Reference : Common GDB Commands

Command	Shorthand	Description / Purpose
break <line / func>	b	Sets a breakpoint at a specified line number or function name.
run	r	Starts execution of the program from the beginning.
next	n	Executes the next line of code (steps over function calls).
step	s	Executes the next line of code (steps into function calls).
print <var>	p	Prints the current value of a variable or expression.

display <var>	disp	Automatically prints the value of a variable after every step.
continue	c	Resumes execution until the next breakpoint or program completion.
list	l	Prints source code lines around the current execution position.
info locals	info loc	Lists all local variables and their current values in the current frame.
backtrace	bt	Displays the current call stack (chain of function calls).
info breakpoints	i b	Displays all active breakpoints and their numbers.
delete <id>	d	Deletes the specified breakpoint number (or all if omitted).
disable <id>	dis	Temporarily disables a breakpoint without deleting it.
enable <id>	en	Re-enables a previously disabled breakpoint.
quit	q	Exits the GDB debugging session.

CASE-4 : Debugging a Logical Error using GDB

```
//Example code(sample4.c)
#include<stdio.h>
int main()
{
    int i,sum=0;
    for(i=0;i<5;i++)
    {
        sum=sum+i;
    }
    printf("Sum=%d\n",sum);
    return 0;
}
```

a. Output

```
kaith@fedora:~/codes/exp_3$ cc sample4.c
kaith@fedora:~/codes/exp_3$ ./a.out
Sum=10
kaith@fedora:~/codes/exp_3$
```

- Expected Output: 15 (1 + 2 + 3 + 4 + 5)
- Actual Output: 10 (1 + 2 + 3 + 4)

b. Debugging Walkthrough in GDB

```
kaith@fedora:~/codes/exp_3$ cc -g sample4.c -o sample4
kaith@fedora:~/codes/exp_3$ gdb ./sample4
GNU gdb (Fedora Linux) 17.2-2.fc44
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-redhat-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./sample4...
(gdb) list
1      #include<stdio.h>
2      int main()
3      {
4          int i,sum=0;
5          for(i=0;i<5;i++)
6          {
7              sum=sum+i;
8          }
9          printf("Sum=%d\n",sum);
10         return 0;
(gdb) break 7
Breakpoint 1 at 0x40047e: file sample4.c, line 7.
(gdb) run
Starting program: /home/kaith/codes/exp_3/sample4

This GDB supports auto-downloading debuginfo from the following URLs:
<ima:enforcing>
<https://debuginfod.fedoraproject.org/>
<ima:ignore>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib64/libthread_db.so.1".

Breakpoint 1, main () at sample4.c:7
7          sum=sum+i;
(gdb) display i
1: i = 0
(gdb) display sum
2: sum = 0
(gdb) continue
Continuing.

Breakpoint 1, main () at sample4.c:7
7          sum=sum+i;
1: i = 1
2: sum = 0
(gdb) continue
Continuing.

Breakpoint 1, main () at sample4.c:7
7          sum=sum+i;
1: i = 2
2: sum = 1
(gdb) continue
Continuing.

Breakpoint 1, main () at sample4.c:7
7          sum=sum+i;
1: i = 3
2: sum = 3
(gdb) continue
Continuing.

Breakpoint 1, main () at sample4.c:7
7          sum=sum+i;
1: i = 4
2: sum = 6
(gdb) continue
Continuing.
Sum=10
[Inferior 1 (process 14587) exited normally]
(gdb)
```

c. Conclusion

When the loop reaches at $i=5$ it exit without executing the loop body, that means the statement should be changed from “ $i<5$ ” to “ $i\leq 5$ ”.

CASE-5 : Debugging Functions (next vs. step)

- next: Executes the line. If the line contains a function call, the function executes entirely in the background without entering its body.
- step: Steps into the function call, transferring GDB's focus into the function body for line-by-line inspection.

```
//Example code(sample5.c)
#include <stdio.h>
int square(int x)
{
    int result;
    result = x * x;
    return result;
}
int main()
{
    int result, n = 5;
    result = square(n);
    printf("Square = %d\n", result);
    return 0;
}
```

a. Method 1 - stepping over with next

```
(gdb) break main
Breakpoint 1 at 0x400483: file sample5.c, line 10.
(gdb) run
Starting program: /home/kaith/codes/exp_3/a.out

This GDB supports auto-downloading debuginfo from the following URLs:
<ima:enforcing>
<https://debuginfod.fedoraproject.org/>
<ima:ignore>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib64/libthread_db.so.1".

Breakpoint 1, main () at sample5.c:10
10      int result, n = 5;
```



```

Starting program: /home/kaith/codes/exp_3/a.out

This GDB supports auto-downloading debuginfo from the following URLs:
  <ima:enforcing>
  <https://debuginfod.fedoraproject.org/>
  <ima:ignore>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib64/libthread_db.so.1".

Breakpoint 1, main () at sample5.c:10
10      int result, n = 5;
(gdb) run
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/kaith/codes/exp_3/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib64/libthread_db.so.1".

Breakpoint 1, main () at sample5.c:10
10      int result, n = 5;
(gdb) next
11      result = square(n);
(gdb) next
12      printf("Square = %d\n", result);
(gdb) print result
$1 = 25

```

Conclusion : In this method we are executing the squares without stepping inside square()

b. Method 2 - stepping into with step

```

(gdb) break main
Breakpoint 1 at 0x400483: file sample5.c, line 10.
(gdb) run
Starting program: /home/kaith/codes/exp_3/a.out

This GDB supports auto-downloading debuginfo from the following URLs:
  <ima:enforcing>
  <https://debuginfod.fedoraproject.org/>
  <ima:ignore>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib64/libthread_db.so.1".

Breakpoint 1, main () at sample5.c:10
10      int result, n = 5;
(gdb) next
11      result = square(n);
(gdb) step
square (x=5) at sample5.c:5
5      result1 = x * x;
(gdb) print x
$1 = 5
(gdb) next
6      return result1;
(gdb) print result1
$2 = 25
(gdb) █

```

Conclusion : In this method we are executing the squares by stepping into fuction calls.

Case 6: Debugging Runtime Errors using Backtrace (backtrace / bt)

When a program crashes (e.g., floating point exception / division by zero), GDB can pinpoint the exact function call sequence leading to the crash.

```
//Example code(sample6.c)
#include <stdio.h>
int divide(int a, int b)
{
    return a / b;
}
int calculate(int x)
{
    return divide(x, 0);
}
int main()
{
    int result = calculate(10);
    printf("Result: %d\n", result);
    return 0;
}
```

a. Output

```
kaith@fedora:~/codes/exp_3$ cc -g sample6.c
kaith@fedora:~/codes/exp_3$ gdb ./a.out
bash: gdb./a.out: No such file or directory
kaith@fedora:~/codes/exp_3$ ls
a.out sample1.c sample2.c sample3.c sample4 sample4.c sample5.c sample6.c
kaith@fedora:~/codes/exp_3$ gdb ./a.out
GNU gdb (Fedora Linux) 17.2-2.fc44
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-redhat-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./a.out...
(gdb) run
Starting program: /home/kaith/codes/exp_3/a.out

This GDB supports auto-downloading debuginfo from the following URLs:
  <ima:enforcing>
  <https://debuginfod.fedoraproject.org/>
  <ima:ignore>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Downloading separate debug info for system-supplied DSO at 0x7ffff7fc2000
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib64/libthread_db.so.1".

Program received signal SIGFPE, Arithmetic exception.
0x0000000000400474 in divide (a=10, b=0) at sample6.c:4
4       return a / b;
(gdb) |
```

```
(gdb) backtrace
#0  0x00000000400474 in divide (a=10, b=0) at sample6.c:4
#1  0x00000000400493 in calculate (x=10) at sample6.c:8
#2  0x000000004004a7 in main () at sample6.c:12
(gdb)
```

- Frame #0: The function where the crash actually happened (divide).
- Frame #1: The function that called line #0 (calculate).
- Frame #2: The top-level function that started the call sequence (main).
- We can also check a particular line or frame using the basic frame command.

Case 7: Debugging Array Out-of-Bounds Errors using GDB

In C, array bounds are not checked at runtime automatically. Accessing indices beyond the declared limit results in undefined behavior or memory corruption.

```
//Example code(sample7.c)
#include <stdio.h>
int main()
{
    int a[5];
    int i;
    for(i = 0; i <= 5; i++)
    {
        scanf("%d", &a[i]);
    }
    return 0;
}
```

a. Output

```
kaith@fedora:~/codes/exp_3$ cc -g sample7.c
kaith@fedora:~/codes/exp_3$ gdb ./a.out
bash: gdb./a.out: No such file or directory
kaith@fedora:~/codes/exp_3$ gdb ./a.out
GNU gdb (Fedora Linux) 17.2-2.fc44
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-redhat-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./a.out...
(gdb) break 8
Breakpoint 1 at 0x400477: file sample7.c, line 8.
(gdb) run
Starting program: /home/kaith/codes/exp_3/a.out

This GDB supports auto-downloading debuginfo from the following URLs:
  <ima:enforcing>
  <https://debuginfod.fedoraproject.org/>
  <ima:ignore>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib64/libthread_db.so.1".

Breakpoint 1, main () at sample7.c:8
8      scanf("%d", &a[i]);
(gdb) print &a[0]
$1 = (int *) 0x7fffffffddce0
(gdb) print &a[4]
$2 = (int *) 0x7fffffffddcf0
(gdb) print &a[5]
$3 = (int *) 0x7fffffffddcf4
(gdb) █
```

- In the `&a[5]` the indices is coming as 0.....4 because 5 is not included in the loop statement so to include the `[5]` the correct loop statement will be “for(i = 0; i < 5; i++)”

Case 8: Segmentation Faults (SIGSEGV) using GDB

A Segmentation Fault occurs when a program attempts to access a memory location that it does not have permission to access (such as dereferencing a NULL pointer).

```
//Example code(sample8.c)
#include <stdio.h>
int main()
{
    int *p = NULL;
    *p = 10;
    printf("%d\n", *p);
    return 0;
}
```

a. Output

```
kaith@fedora:~/codes/exp_3$ cc -g sample8.c
kaith@fedora:~/codes/exp_3$ gdb ./a.out
GNU gdb (Fedora Linux) 17.2-2.fc44
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-redhat-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./a.out...
(gdb) run
Starting program: /home/kaith/codes/exp_3/a.out

This GDB supports auto-downloading debuginfo from the following URLs:
    <ima:enforcing>
    <https://debuginfod.fedoraproject.org/>
    <ima:ignore>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib64/libthread_db.so.1".

Program received signal SIGSEGV, Segmentation fault.
0x000000000040047a in main () at sample8.c:5
5      *p = 10;
(gdb) █
```

- As we haven't a memory location to the pointer *p that's why we are unable to assign it's location to "10"

5. Exercises & Homework Questions

5.1 Error Classification

- Syntax Error

```
#include <stdio.h>
int main( )
{
    int x = 10                //4
    printf("%d\n", x);
    return 0;
}
```

Missing semicolon at the end of the 4th line.

- Compilation Error

```
#include <stdio.h>
int main(void)
{
    int *ptr = 100;
    printf("%p\n", (void*)ptr);
    return 0;
}
```

Incompatible pointer-to-integer conversion error
/ type mismatch

- Linker Error

```
void calculateTotal(void);
int main(void)
{
    calculateTotal();
    return 0;
}
```

// Triggers: undefined reference to
`calculateTotal`

- Runtime Error

```
#include <stdio.h>
int main(void)
{
    int a = 10; int b = 0;
    int result = a / b;
    printf("Result: %d\n", result);
    return 0;
}
```

- // Division by zero causes the OS to terminate the program.

- Logical Error

```
#include<stdio.h>
int main()
{
    int i,sum=0;
    for(i=0;i<5;i++)
    {
        sum=sum+i;
    }
    printf("Sum=%d\n",sum);
    return 0;
}
```

- Program terminates at i=4 because of the wrong loop statement.

5.2 GNU Concept

The GNU Project

Launched by Richard Stallman in 1983, the GNU Project (a recursive acronym for "GNU's Not Unix") was initiated to develop a complete, free, and open-source Unix-compatible operating system.

- **Philosophy:** It established the Free Software movement, emphasizing users' freedom to run, study, modify, and redistribute software, formalized through the GNU General Public License (GPL).
- **Core Components:** GNU developed the essential user-space components of an operating system—including the compiler toolchain (GCC), core utilities (coreutils), the standard C library (glibc), and the command-line shell (Bash). Combined with the Linux kernel in the early 1990s, it forms the full operating system commonly known as GNU/Linux.

GDB (GNU Debugger) is the flagship, standard source-level debugger within the GNU software development toolchain.

- **Toolchain Integration:** It works alongside GCC (GNU Compiler Collection). When source code is compiled with debugging symbols enabled (using the `-g` flag), GCC embeds metadata (such as DWARF format tables) that GDB interprets to map machine execution directly back to human-readable source code.
- **Debugging Capabilities:** GDB allows developers to inspect program execution line-by-line, set breakpoints/watchpoints, inspect memory and CPU registers, analyze stack frames, and diagnose runtime crashes (such as segmentation faults via core dump post-mortem analysis).
- **Cross-Platform Role:** In the broader GNU ecosystem, GDB provides a unified debugging interface across numerous architectures (x86, ARM, RISC-V, MIPS) and supports multiple languages including C, C++, Rust, and Fortran.

5.3 Shell Context

Kernel vs. Shell

- **Kernel:** The core, low-level component of an operating system that runs with complete hardware privileges. It directly manages hardware resources—such as CPU scheduling, memory allocation, storage I/O, and device drivers—and provides system calls for applications to interact safely with the hardware.
- **Shell:** A user-space program that acts as the command-line interface (CLI) between the user and the kernel. It reads user input (or scripts), parses the text, translates it into system requests or executable process launches, and returns the output.

Three Common Linux Shells

- **Bash (Bourne Again Shell):** The default standard shell on most Linux distributions. Highly compatible with the original Bourne Shell (sh), widely used for system scripting, automation, and command-line management.
- **Zsh (Z Shell):** An extended, interactive shell incorporating features from Bash, ksh, and tcsh. Known for advanced tab completion, themeable prompt frameworks (like Oh My Zsh), and customizable plugins.
- **Fish (Friendly Interactive Shell):** A modern, user-centric shell designed for out-of-the-box ease of use. It features syntax highlighting as you type, autosuggestions based on command history, and web-based configuration, though it deliberately diverges from strict POSIX scripting compliance.

5.4 Research and contrast the differences between: Breakpoint, Watchpoint, Catchpoint, and Tracepoint.

Definitions & Contrasts

- Breakpoint
 - Trigger: Set on a specific source code line, function name, or instruction memory address.
 - Execution behavior: Pauses execution immediately before the target instruction runs, handing interactive control to the debugger.
 - Use case: Stepping through a function line-by-line to inspect local variables and control flow.
 - Watchpoint (Data Breakpoint)
 - Trigger: Set on a variable, memory address, or data expression.
 - Execution behavior: Pauses execution whenever the watched memory is written to, read from, or modified (often leveraging hardware debug registers).
 - Use case: Pinpointing exactly which line of code corrupts a pointer, array, or global variable.
 - Catchpoint
 - Trigger: Set on high-level system or runtime events (e.g., C++ exceptions thrown/caught, dynamic library loading via dlopen, fork, or signal reception).
 - Execution behavior: Pauses execution the instant the specified event occurs.
 - Use case: Trapping an unhandled exception or inspecting program state the moment a shared library is linked.
 - Tracepoint
 - Trigger: Set on a specific code location, similar to a breakpoint.
 - Execution behavior: Does not pause program execution. It silently evaluates expressions and logs designated variables/registers into a trace buffer as the program continues running at full speed.
 - Use case: Debugging timing-sensitive issues, race conditions, or real-time embedded systems where halting execution would change runtime behavior.
-